

Bases de Datos

Clase 18: Transacciones — ACID, Anomalías y Serializabilidad

Lunes 11 de mayo

Hasta ahora... estamos solos

- Hemos estudiado el DBMS asumiendo **un único usuario** ejecutando una consulta a la vez.
- Storage, índices, joins, optimizador → todo razonado para un plan que corre solo.
- En la realidad, **un DBMS atiende a miles de usuarios concurrentemente.**
- ¿Qué cambia?

Agenda

1. Motivación: el problema de la concurrencia
2. Transacciones y propiedades **ACID**
3. Anomalías de concurrencia
4. **Schedules** y serializabilidad
5. Locks y **Two-Phase Locking**

Problema: una transferencia bancaria

```
-- Transferir v unidades de la cuenta 1 a la cuenta 2
```

```
UPDATE cuentas SET saldo = saldo - v WHERE cid = 1;
```

```
UPDATE cuentas SET saldo = saldo + v WHERE cid = 2;
```

Vista programáticamente (lo que el DBMS realmente ejecuta):

```
READ(saldo1, x);  WRITE(saldo1, x - v);
```

```
READ(saldo2, y);  WRITE(saldo2, y + v);
```

¿Qué pasa si **dos** transferencias se ejecutan al mismo tiempo?

Caso: Alice y Bob comparten cuenta

- Cuenta **AB** tiene 1000. Cuenta de Charles **C** tiene 1000.
- **Alice** transfiere 100 desde **AB** a **C**.
- **Bob** transfiere 200 desde **AB** a **C**.
- Resultado esperado (cualquiera sea el orden): **AB = 700** , **C = 1300**.

¿Qué puede salir mal si las dos transacciones se mezclan?

Falla 1: Lost update

Alice	Bob	AB	C
READ(AB, x) $\rightarrow$ 1000		1000	1000
	READ(AB, y) $\rightarrow$ 1000	1000	1000
WRITE(AB, x - 100)		900	1000
	WRITE(AB, y - 200)	800	1000
READ(C, x) $\rightarrow$ 1000		800	1000
WRITE(C, x + 100)		800	1100
	READ(C, y) $\rightarrow$ 1100	800	1100
	WRITE(C, y + 200)	800	1300

AB = 800 $\rightarrow$ **el banco perdió 100**. Bob leyó el saldo antes de que Alice escribiera.

Falla 2: dirty write

Alice	Bob	AB	C
READ(AB, x)		1000	1000
WRITE(AB, x - 100)		900	1000
READ(C, x)		900	1000
	READ(AB, y)	900	1000
	WRITE(AB, y - 200)	700	1000
	READ(C, y) → 1000	700	1000
	WRITE(C, y + 200)	700	1200
WRITE(C, x + 100)		700	1100

C = 1100 → **Charlie perdió 200**. Alice leyó **C** antes de que Bob lo actualizara, y al

escribir escribió la actualización de Bob.

Falla 3: error a mitad de transacción

Alice	AB	C
READ(AB, x) $\rightarrow$ 1000	1000	1000
WRITE(AB, x - 100)	900	1000
ERROR / corte de luz	900	1000

La cuenta **AB** quedó descontada **sin que C recibiera el dinero.**

→ Necesitamos garantías que la BD por sí sola no nos da.

Diagnóstico

Lo **ideal**: cada transferencia se ejecuta de principio a fin antes de que empiece la siguiente.

Lo **real**: el DBMS intercala operaciones para **maximizar throughput** (mientras una transacción espera I/O, otra puede avanzar).

Pregunta: ¿cómo intercalar operaciones **sin errores**?

Transacciones y propiedades ACID

¿Qué es una transacción?

Una **transacción** es una secuencia de una o más operaciones (lecturas y/o escrituras) que el DBMS trata como una **unidad lógica**.

Ejemplos:

- Transferencia de dinero entre cuentas.
- Compra por internet (cargar tarjeta + reducir stock + emitir factura).
- Inscripción de un curso (insertar fila + reducir cupos).
- Reservar un asiento de avión.

Transacciones en SQL

```
START TRANSACTION;    -- o BEGIN

UPDATE cuentas SET saldo = saldo - 100 WHERE cid = 1;
UPDATE cuentas SET saldo = saldo + 100 WHERE cid = 2;

COMMIT;               -- confirma la transacción
-- ROLLBACK;          -- la deshace
```

- `START TRANSACTION` / `BEGIN` abre la transacción.
- `COMMIT` la confirma: los cambios son **definitivos**.
- `ROLLBACK` la aborta: la BD vuelve al estado previo, **como si nunca hubiera ocurrido**.

En PostgreSQL, fuera de un bloque explícito **cada sentencia es su propia transacción** (autocommit).

ACID: 4 garantías

Propiedad	Significado
Atomicity	La transacción se ejecuta completa o no se ejecuta
Consistency	Lleva la BD de un estado consistente a otro consistente
Isolation	Las transacciones concurrentes no se interfieren
Durability	Los cambios confirmados persisten ante fallas

Una transacción es la unidad sobre la que el DBMS garantiza ACID.

Atomicity ("todo o nada")

O se ejecutan **todas** las operaciones de la transacción, o **ninguna**.

Si Alice escribió en **AB** pero falla antes de escribir en **C** :

- El sistema debe **deshacer** la escritura en **AB** .
- Nadie debe ver el estado intermedio.

Sin atomicity → falla 3 (cuenta descontada sin acreditar).

Consistency

Cada transacción **preserva las reglas** de la BD: claves, FKs, CHECKs, invariantes de negocio.

Antes: $AB + C = 2000$. Después: $AB + C = 2000$.

- Es responsabilidad **conjunta** del programador (escribir la lógica correcta) y del DBMS (chequear las restricciones declarativas).
- Ojo: el DBMS no sabe que el invariante "el total no cambia" debe mantenerse — eso lo asegura el programador *agrupando las dos updates en una transacción*.

Isolation

Cada transacción se ejecuta **como si estuviera sola**, aunque en realidad se intercale con otras.

Sin isolation → fallas 1 y 2 (lost update, doble cobro).

Podemos lograrlo fácilmente ejecutando en serie, pero sería **lentísimo**: 1 transacción a la vez bloquea todo el sistema.

→ El reto es permitir intercalado **manteniendo la ilusión** de ejecución secuencial.

Durability

Una vez que el sistema confirma `COMMIT`, los cambios **persisten** ante cualquier falla posterior (corte de luz, crash del proceso, reinicio del sistema).

Implementación → **logs** y **recovery** (próxima semana).

Sin durability → un cliente recibe "transferencia exitosa" y al rato el dinero "se desaparece".

¿Quién garantiza qué?

Transaction Manager → Atomicity, Isolation
Log & Recovery Manager → Atomicity, Durability
Constraint Checker → Consistency (PK, FK, CHECK)

- **Esta clase y la siguiente:** Isolation (conurrencia, locks).
- **Semana 13:** Atomicity y Durability (log y recovery).

Paréntesis: Jim Gray (Turing 1998)

Premios Turing relevantes en BD:

- **1973** — Charles Bachman, primeros DBMS.
- **1981** — Edgar Codd, modelo relacional.
- **1998** — **Jim Gray, transacciones.**
- **2014** — Michael Stonebraker, Ingres / Postgres.

Gray formalizó las propiedades ACID y los conceptos de bloqueo y serializabilidad que veremos hoy y la próxima clase.

Anomalías de concurrencia

Tres tipos de conflictos entre transacciones

Considerando dos transacciones T_i, T_j que tocan el mismo objeto X :

Conflicto	Patrón	Anomalía típica
Write – Read	$W_i(X) \dots R_j(X)$	Dirty read
Read – Write	$R_i(X) \dots W_j(X)$	Non-repeatable read
Write – Write	$W_i(X) \dots W_j(X)$	Lost update

Si dos operaciones sobre el **mismo objeto** y **al menos una es escritura**, hay conflicto: **el orden importa**.

Anomalía 1: Dirty Read (W → R)

T2 lee un valor que T1 escribió pero **aún no confirmó**.

T1	T2	A
READ(A, x)		1000
WRITE(A, x – 100)		900
	READ(A, y) → 900	900
	WRITE(A, y × 1.1)	990
ROLLBACK (vuelve A a 1000)		990 

T1 abortó pero T2 ya **trabajó con datos sucios**. La BD queda inconsistente.

Anomalía 2: Non-Repeatable Read (R → W)

T1 lee **A** dos veces y obtiene **valores distintos**, porque T2 lo modificó entre medio.

T1	T2	A
READ(A, x) → 1		1
IF x > 0		1
	READ(A, y) → 1	1
	WRITE(A, y - 1)	0
	COMMIT	0
WRITE(A, x - 1) → -1		-1 

T1 razonó "A es 1, le resto 1, queda 0". Pero T2 ya lo había bajado: termina en **-1**, violando un check.

Anomalía 3: Lost Update (read-modify-write)

El patrón canónico de la **Falla 1** que vimos al comienzo: dos transacciones leen el mismo valor, lo modifican, y escriben — la segunda escritura **aplasta** la primera.

Alice	Bob	AB
READ(AB, x) → 1000		1000
	READ(AB, y) → 1000	1000
WRITE(AB, x – 100)		900
	WRITE(AB, y – 200)	800 

Bob escribió $1000 - 200 = 800$ ignorando los -100 de Alice. La actualización de Alice **se perdió**: el saldo final debería ser 700.

Anomalía 4: Phantom Read

Aparece cuando una transacción **lee un conjunto** y otra **inserta o borra** filas que satisfacen su predicado.

```
-- T1, primera lectura
SELECT COUNT(*) FROM Reservas WHERE bid = 103;    -- 5

-- T2 inserta una nueva reserva en bid = 103 y hace commit

-- T1, segunda lectura (misma transacción)
SELECT COUNT(*) FROM Reservas WHERE bid = 103;    -- 6  
```

Mismo predicado, distinto resultado. El "fantasma" es la fila nueva que T1 nunca leyó pero ahora aparece.

Resumen de anomalías

Anomalía	Patrón	Idea
Dirty read	$W \rightarrow R$	Leí un dato no confirmado
Non-repeatable read	$R \rightarrow W$	Releí y cambió
Lost update	$W \rightarrow W$	Mi escritura sobrescribió la tuya
Phantom read	Inserciones / borrados	Mi predicado retorna distintos conjuntos

Necesitamos un criterio formal para decir "**este intercalado de operaciones es seguro**" y otro para decir "**este no lo es**".

Schedules y Serializabilidad

Notación

Modelaremos cada transacción como una secuencia de operaciones primitivas:

Símbolo	Significado
$R_i(X)$	Transacción i lee el objeto X
$W_i(X)$	Transacción i escribe el objeto X
C_i	Transacción i hace commit
A_i	Transacción i aborta

Ignoramos los cálculos en RAM; modelamos sólo lo que el DBMS ve.

Schedule

Un **schedule** S es una secuencia de operaciones de una o más transacciones, tal que para cada transacción, sus acciones aparecen en S **en el mismo orden** que las definió.

Ejemplo:

```
S = R_1(A)  W_1(A)  R_2(A)  W_2(A)  R_1(B)  W_1(B)  R_2(B)  W_2(B)
```

Las operaciones de T1 y T2 se intercalan, pero **dentro de cada transacción** mantienen su orden original.

Schedule **serial**

Un schedule es **serial** si **no hay intercalado**: las operaciones de cada transacción aparecen contiguas.

$S_{\text{serial}} = \underbrace{R_1(A) \ W_1(A) \ R_1(B) \ W_1(B)}_{T1} \ \underbrace{R_2(A) \ W_2(A) \ R_2(B) \ W_2(B)}_{T2}$

- Para n transacciones hay $n!$ schedules seriales (un orden por permutación).
- **Cada uno** es trivialmente correcto: corren una a la vez.

Schedule **serializable**

Un schedule S es **serializable** si existe algún schedule serial S' con las mismas transacciones tal que el resultado de S y S' es **el mismo** sobre todo estado inicial.

La meta del Transaction Manager:

Permitir sólo schedules serializables (o equivalentemente: rechazar / posponer operaciones que romperían serializabilidad).

Esto asegura **isolation**: la ejecución concurrente es **observacionalmente equivalente** a alguna ejecución secuencial.

 **Serializable $\neq$ serial.** Un schedule serial **no tiene intercalado**; uno serializable sí **tiene intercalado**, pero es equivalente *en resultado* a alguno serial. Lo segundo es lo que queremos: concurrencia real con la *ilusión* de ejecución secuencial.

Ejemplo: serializable vs. no serializable

T1: $A := A + 100$, luego $B := B + 100$ (mantiene $A - B$).

T2: $A := A \times 2$, luego $B := B \times 2$ (mantiene A / B).

Schedule serializable (equivalente a T1; T2):

$R_1(A) \ W_1(A) \ R_1(B) \ W_1(B) \ R_2(A) \ W_2(A) \ R_2(B) \ W_2(B)$

Schedule no serializable:

$R_1(A) \ W_1(A) \ R_2(A) \ W_2(A) \ R_2(B) \ W_2(B) \ R_1(B) \ W_1(B)$

T2 ve el A actualizado pero B viejo $\rightarrow$ no equivale a "T1; T2" ni a "T2; T1".

Acciones **conflictivas**

Para dos transacciones distintas $i \neq j$, son **conflictivas** y no se pueden permutar libremente:

- $R_i(X), W_j(X)$
- $W_i(X), R_j(X)$
- $W_i(X), W_j(X)$

Y **no son conflictivas** (sí se pueden permutar):

- Dos lecturas del mismo objeto: $R_i(X), R_j(X)$.
- Cualquier par sobre **objetos distintos**.

Regla: sobre el mismo objeto, si **al menos una** es escritura $\rightarrow$ conflicto.

Conflict-serializable

Un schedule es **conflict-serializable** si puede transformarse en un schedule **serial** mediante permutaciones de pares **adyacentes no conflictivos**.

Es una definición **operacional**: dame el schedule, intento "ordenarlo" por intercambios legales hasta dejar las transacciones contiguas.

... R_1(A) R_2(B) ...	← objetos distintos, intercambiables
... R_1(A) W_2(A) ...	← mismo objeto + escritura, NO intercambiable

Conflict-serializable $\subseteq$ Serializable

- Si un schedule es conflict-serializable, entonces es serializable. (todo intercambio legal preserva el resultado)
- El recíproco **no vale**: hay schedules serializables (equivalentes a uno serial por *valor*) que **no** son conflict-serializables.

Conflict-serializable es la noción **operacional** que usan los DBMS: se decide rápido (lo veremos a continuación) y es lo suficientemente fuerte para implementarla en runtime.

Grafo de precedencia

Dado un schedule S , su **grafo de precedencia** $P(S)$:

- **Nodos:** las transacciones de S .
- **Arista** $T_i \rightarrow T_j$ si en S aparece una operación de T_i **antes** de una operación de T_j que **conflictúa** con ella.

Intuición: la arista dice " T_i debe estar antes que T_j en cualquier schedule serial equivalente".

Teorema (Eswaran et al., 1976)

Un schedule es **conflict-serializable** si y sólo si su grafo de precedencia es **acíclico**.

Algoritmo:

1. Construir $P(S)$ recorriendo S y agregando aristas por cada par conflictivo.
2. Verificar que $P(S)$ no tenga ciclos (DFS o orden topológico).
3. Si no tiene ciclos $\rightarrow$ cualquier orden topológico **es** un schedule serial equivalente.

Costo: $O(|S| + n^2)$ para n transacciones.

Ejemplo paso a paso (pizarra)

$S = R_2(A) \ R_1(B) \ W_2(A) \ R_3(A) \ W_1(B) \ W_3(A) \ R_2(B) \ W_2(B)$

Buscar pares conflictivos:

- $R_2(A) \dots W_3(A) \rightarrow$ arista $T_2 \rightarrow T_3$
- $W_2(A) \dots R_3(A) \rightarrow$ arista $T_2 \rightarrow T_3$ (ya está)
- $R_3(A) \dots W_3(A) \rightarrow$ misma transacción, no cuenta
- $R_1(B) \dots W_2(B) \rightarrow$ arista $T_1 \rightarrow T_2$
- $W_1(B) \dots R_2(B) \rightarrow$ arista $T_1 \rightarrow T_2$ (ya está)

Grafo: $T_1 \rightarrow T_2 \rightarrow T_3$. **Acíclico** $\rightarrow$ es conflict-serializable. Equivalente serial: $T_1; T_2; T_3$.

Del análisis a la prevención

Ya sabemos **detectar** si un schedule es serializable a posteriori (grafo de precedencia).

Pero el Transaction Manager debe **prevenir** schedules incorrectos **en tiempo real**, mientras las transacciones se ejecutan, sin ver el futuro.

→ Necesitamos un **protocolo** que, al ejecutarlo, sólo permita schedules conflict-serializables.

Locks y Two-Phase Locking

La idea: pedir permiso antes de tocar un objeto

- Antes de leer o escribir un objeto X , la transacción **adquiere un lock** sobre X .
- Si otra transacción tiene un lock incompatible, **espera**.
- Cuando termina, **libera** sus locks.

Los locks son el mecanismo que **fuerza un orden** sobre los accesos conflictivos.

Dos tipos de lock

Lock	Símbolo	Permite
Shared (lectura)	S(X)	Que otras también lean X
Exclusive (escritura)	X(X)	Nadie más accede a X

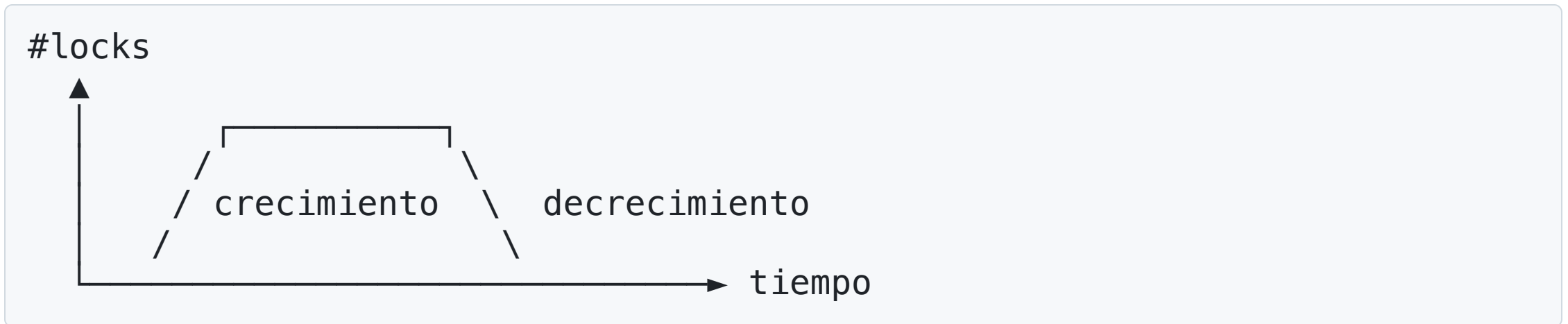
Tabla de compatibilidad:

	S	X
S	✓	×
X	×	×

Múltiples lectores **sí**. Una escritora **sola**.

Two-Phase Locking (2PL)

Una transacción sigue **2PL** si todas sus operaciones de **adquirir** preceden a todas sus operaciones de **liberar** locks.



Una vez que liberaste el primer lock, **no puedes pedir más**.

Garantía fundamental

Teorema: si todas las transacciones siguen 2PL, todo schedule resultante es **conflict-serializable**.

Idea: el orden serial equivalente está dado por los *puntos de bloqueo* (instante en que cada transacción tiene su último lock). 2PL hace que el grafo de precedencia respete ese orden → no hay ciclos.

2PL en acción: el caso Alice / Bob

Volvamos al schedule sin 2PL que perdía 100 (Falla 1):

```
Alice: R(AB)  W(AB)  R(C)  W(C)
Bob:      R(AB)  W(AB)  R(C)  W(C)
```

Grafo de precedencia: Alice → Bob (W(AB) → R(AB), W(C) → R(C)) y Bob → Alice (R(AB) → W(AB)) → **ciclo** → no serializable.

Con 2PL (locks adquiridos antes de cada operación):

```
Alice: X(AB) R(AB) W(AB) X(C) R(C) W(C) release todo
Bob:   X(AB) ←—— ESPERA —————→ X(AB) R(AB) W(AB) ...
```

Bob no obtiene X(AB) hasta que Alice libera → el schedule resultante es equivalente a "Alice; Bob". El ciclo del grafo de precedencia **desaparece** porque la arista Bob → Alice ya no se forma.

En síntesis (clase de hoy)

- Una **transacción** es la unidad protegida con **ACID**.
- La concurrencia genera anomalías; un schedule es **seguro** si es serializable.
- **Conflict-serializable** se decide en tiempo polinomial vía el grafo de precedencia.
- **2PL** asegura serializabilidad **en runtime** mediante locks en dos fases.

Próxima clase

2PL parece resolverlo todo, pero es **más sutil** de lo que parece:

- ¿Qué pasa si una transacción libera locks **antes del commit** y luego aborta? → **cascading aborts** y **Strict 2PL**.
- ¿Qué pasa si dos transacciones se esperan mutuamente? → **deadlocks**: detección y prevención.
- ¿Realmente necesitamos *siempre* serializable? → **niveles de aislamiento**, write skew, y por qué `REPEATABLE READ` en PostgreSQL **no** es serializable.
- ¿Cómo expresar reglas activas que se ejecutan **dentro** de la transacción? → **triggers**.